AJUDA PASSO A PASSO

CRUD READ — JavaScript e Node.js

Guia passo a passo para estudantes | 2 dias de 5 aulas de 45 minuto

Nome: Maria Eduarda Sampaio Assumpção Turma: 2DEVIS-SESI

OBJETIVO: aprender a abrir um arquivo JSON, mostrar máquinas no terminal e consultar informações usando JavaScript. Não é necessário criar um projeto do zero.

Antes de começar: prepare a pasta

☐ No Classroom, baixe o arquivo Projeto_Aluno_READ.zip.

☐ Na pasta Downloads, clique com o botão direito no ZIP e escolha “Extrair tudo”.

☐ Abra a pasta extraída Projeto_Aluno. Confira estes arquivos: app.js, maquinas.json, package.json, testes.js e README.txt.

☐ Abra o VS Code. Clique em Arquivo > Abrir Pasta e selecione Projeto_Aluno.

☐ No VS Code, clique em Terminal > Novo Terminal. Confira se o caminho do terminal termina em Projeto_Aluno.

☐ Digite node --version e pressione Enter. Se aparecer v... (uma versão), o Node.js está instalado.

☐ Digite npm start e pressione Enter. A mensagem inicial mostra que o programa abriu, mas as funções ainda precisam ser implementadas.

Combinados importantes

- Não apague os arquivos maquinas.json, testes.js ou package.json.
- Edite somente app.js. Salve sempre com Ctrl + S.
- Atenção: em maquinas.json, o campo é status (não situacao). O valor é manutencao, sem acento e com letras minúsculas.
- Se algo der errado, leia a mensagem do terminal, confira a linha e compare com o manual.

DIA 1 — Fazer o programa ler e listar (5 aulas)

Passo 1 — Conheça os dados (aula 1)

No VS Code, clique no arquivo maquinas.json. Ele contém uma LISTA de 8 máquinas. Cada máquina possui id, nome, setor, status e horasUso.

Encontre a máquina com id 3. Anote o nome: Esteira-E

Encontre quantas máquinas estão com status "manutencao": 2 Máquinas, a prensa hidráulica P-02 e a P-06.

Passo 2 — Carregar o JSON (aulas 2 e 3)

Clique em app.js. Não remova as linhas const fs, const caminho e const ARQUIVO no início. Localize a função carregarMaquinas(). Apague apenas os comentários TODO 1 a TODO 3 e o throw new Error dentro dela. Deixe a função assim:

```
function carregarMaquinas() {
  const texto = fs.readFileSync(ARQUIVO, 'utf8');
  const maquinas = JSON.parse(texto);
  return maquinas;
}
```

O que cada linha faz? readFileSync lê o texto do arquivo; JSON.parse converte o texto em dados JavaScript; return entrega a lista à função que a chamou.

Salve com Ctrl + S. No terminal, digite npm test. O primeiro teste deve mostrar OK - Carregar 8 máquinas. Os outros ainda podem falhar: isso é esperado!

Passo 3 — Listar máquinas (aulas 4 e 5)

Localize listarMaquinas(maquinas). Remova o comentário TODO 4 e o throw dela. Digite:

```
function listarMaquinas(maquinas) {
  maquinas.forEach(maquina => {
    console.log(maquina.id + ' - ' + maquina.nome +
      ' - ' + maquina.status);
  });
}
```

Agora localize executar(). Logo DEPOIS da linha console.log já existente, adicione estas duas linhas:

```
  const maquinas = carregarMaquinas();
  listarMaquinas(maquinas);
```

Salve (Ctrl + S) e rode npm start. Esperado: título do sistema e 8 linhas com id, nome e status. Se vir isso, registre uma captura de tela do terminal no relatório.

- ☐ O terminal mostrou 8 máquinas.
- ☐ Salvei o arquivo app.js.
- ☐ Registrei uma captura de tela no relatório.

DIA 2 — Encontrar e filtrar máquinas (5 aulas)

Passo 4 — Procurar pelo ID (aulas 1 e 2)

Assista ao vídeo “CRUD READ na Prática” no NotebookLM. Depois, em app.js, substitua o conteúdo da função buscarPorId (somente dentro das chaves) por:

```
function buscarPorId(maquinas, id) {
  return maquinas.find(maquina => maquina.id === id);
}
```

Na função executar(), depois de listarMaquinas(maquinas), acrescente:

```
  console.log('\nBusca por ID 3:');
  console.log(buscarPorId(maquinas, 3));
  console.log('Busca por ID 99:');
  console.log(buscarPorId(maquinas, 99));
```

Salve e execute npm start. O ID 3 deve apresentar a máquina “Esteira E-03”. O ID 99 deve mostrar undefined (não encontrada). Isso não é erro de leitura.

Passo 5 — Filtrar por status (aulas 3 e 4)

Agora faça um desafio com ajuda: localize filtrarPorStatus(maquinas, status), remova o throw e complete o espaço em branco.

```
function filtrarPorStatus(maquinas, status) {
  return maquinas.filter(maquina => maquina._________ === status);
}
```

DICA: olhe o nome da propriedade dentro do arquivo maquinas.json. Depois de completar, acrescente no FINAL de executar():

```
  console.log('Maquinas em manutencao:');
  console.log(filtrarPorStatus(maquinas, 'manutencao'));
```

Execute npm start. Esperado: array contendo as máquinas de id 2 e 6.

Passo 6 — Desafio final: filtrar setor (aula 5)

Localize filtrarPorSetor(maquinas, setor). Copie a lógica usada no passo 5 e troque o campo pesquisado: agora queremos setor. Dica: nomes como "Usinagem" precisam respeitar as letras maiúsculas.

Depois de implementar, execute npm test. O resultado esperado é 5 linhas iniciadas por OK (um teste por verificação). Se ainda houver FALHOU, compare sua função com o passo anterior.

AJUDA RÁPIDA — Se aparecer erro, faça isto

Mensagem / sintoma	O que conferir
Cannot find module ...	Confira se o terminal está aberto na pasta Projeto_Aluno; confira se os arquivos estão nela.
ENOENT: no such file or directory	Confira se maquinas.json está na mesma pasta de app.js; não altere a variável ARQUIVO.
Unexpected token ... in JSON	Não altere o arquivo JSON. Confira vírgulas e aspas se você o modificou.
Implementar buscarPorId / filtrar...	Ainda existe um throw new Error nessa função. Volte ao passo correspondente.
maquinas.forEach is not a function	Confira se carregarMaquinas() retorna o resultado do JSON.parse(texto).
Mostrou undefined no ID 99	Está correto: não há máquina com esse ID.
Filtro retorna []	Confira status "manutencao" (sem acento) e setor "Usinagem" (maiúscula).
SyntaxError em app.js	Confira parênteses (), chaves {}, aspas e ponto e vírgula. Copie exatamente os exemplos.

Checklist final antes de entregar

☐ Meu projeto abre no VS Code e não alterei o JSON original.

☐ Dia 1: npm start apresenta as 8 máquinas.

☐ Dia 2: consultei ID 3 e ID 99; filtrei manutencao e Usinagem.

☐ Executei npm test e anotei o resultado (inclusive se algum teste falhou).

☐ Preenchi o Modelo de Relatório e colei capturas do terminal.

☐ Exportei meu relatório em PDF e anexei no Classroom.

☐ Anexei app.js no Dia 1; no Dia 2, entreguei o projeto ZIP completo sem node_modules.

Perguntas curtas para o relatório

1. Para que serve JSON.parse()?
   Serve para transformar um texto no formato JSON em um objeto ou array que o JavaScript consegue utilizar.
2. Qual a diferença entre find() e filter()?

   O find() retorna o primeiro elemento que atende à condição, enquanto o filter() retorna um array com todos os elementos que atendem à condição.

3. Por que ID 99 retorna undefined?

   Porque não existe nenhuma máquina com o ID 99 no arquivo maquinas.json, então o find() não encontra nenhum resultado.

4. O que foi mais difícil de corrigir?

   O mais difícil foi corrigir as funções e garantir que retornassem os resultados corretamente, permitindo listar e filtrar as máquinas no terminal.

Se travar: releia o passo, consulte o manual prático e faça uma pergunta específica no NotebookLM. Seu objetivo é entender o que digitou.